

EE147 Exam 2 — Study Guide

Lectures 8–14: Unified Memory, Tiled SGEMM, Coalescing, Histogram, Multi-GPU, MPS, Dynamic Parallelism

Kaushik Vada

Exam: June 4, 2026 • Bourns A125

How to use this guide

Each section maps to one lecture. Read the **Core Idea** first, then the mechanics. The **key facts boxes** are exam-likely. The **gotcha boxes** are common trip-ups. Code snippets are real CUDA — read them carefully.

1 Lecture 8 — Unified Memory (UM)

Core idea

Before UM: you maintain two separate pointers (**h_A** on CPU, **d_A** on GPU) and manually `cudaMemcpy` between them. With UM, you allocate *one* pointer with `cudaMallocManaged` that is accessible from both CPU and GPU. The CUDA runtime moves data automatically.

Two regimes, separated by GPU generation:

- **Pre-Pascal (Kepler/Maxwell)**: entire managed allocation is migrated en-masse at kernel launch. `cudaDeviceSynchronize()` brings it back to CPU. No oversubscription, no concurrent CPU/GPU access, no on-demand migration.
- **Pascal+ (Pascal/Volta/Ampere/...)**: true demand paging. Page faults trigger migration of individual pages. Allowed: oversubscription (allocate > GPU memory), concurrent CPU/GPU access, system-wide atomics.

Code patterns

```
// Without UM (old way):
char *data, *d_data;
data = (char*)malloc(N);
cudaMalloc(&d_data, N);
cudaMemcpy(d_data, data, N, H2D);
qsort<<<...>>>(d_data, N, ...);
cudaMemcpy(data, d_data, N, D2H);
use_data(data);
free(data); cudaFree(d_data);

// With UM (new way):
char *data;
cudaMallocManaged(&data, N);
fread(data, 1, N, fp);
qsort<<<...>>>(data, N, 1, compare);
cudaDeviceSynchronize();
use_data(data);
cudaFree(data);
```

How demand paging actually works

GPU touches a page that isn't resident → **page fault** → driver migrates the page from CPU memory (or other GPU) to local GPU memory → access replays. Many faults = slow.

Performance tuning APIs (Pascal+)

Prefetching — explicitly move data ahead of time:

```
cudaMemPrefetchAsync(ptr, length, destDevice, stream);
// destDevice can also be cudaCpuDeviceId
```

Hints — `cudaMemAdvise(ptr, count, hint, device)`. Hints don't move data by themselves.

- `cudaMemAdviseSetReadMostly` — runtime keeps a local read-only copy on each processor that touches it. A write collapses all copies into one and re-faults subsequent reads.
- `cudaMemAdviseSetPreferredLocation` — suggests best home for data. P2P mapping if possible; otherwise migration on fault.
- `cudaMemAdviseSetAccessedBy` — gives the indicated processor a direct (P2P) mapping without page faults. Does not move data.

System-wide atomics

`atomicAdd_system(addr, val)` on Pascal+ — atomic across CPU and *all* GPUs. Direct over NVLink, software-assisted over PCIe. Not on Kepler/Maxwell.

Pre-Pascal gotcha: touching managed memory on CPU while a kernel runs → **segfault**. On Pascal+ it's just a page fault, but UM gives no ordering / visibility guarantees without explicit sync or system-wide atomics.

Use cases where UM shines

Deep copy (struct with pointer-to-buffer), linked lists, C++ classes (overload `new/delete` to call `cudaMallocManaged`), sparse graph algorithms with unpredictable access patterns.

UM final words

UM is about **programmability**, not making well-tuned code faster. Expertly written manual data movement usually beats UM. Misuse can dramatically slow code (think: kernel that page-faults page-by-page).

2 Lecture 9 — Matrix Multiplication & Tiling

Core idea

We compute $P = M \times N$ where each is a `Width×Width` matrix. Each thread computes one element of P by walking row of M and column of N . Basic kernel = too many redundant global memory reads. **Tiling** = cooperatively load a tile into shared memory, reuse it, then move to next tile.

Basic kernel

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, int Width) {
    int Row = blockIdx.y*blockDim.y + threadIdx.y;
    int Col = blockIdx.x*blockDim.x + threadIdx.x;
    if (Row < Width && Col < Width) {
        float Pvalue = 0;
        for (int k = 0; k < Width; ++k)
            Pvalue += M[Row*Width + k] * N[k*Width + Col];
        P[Row*Width + Col] = Pvalue;
    }
}
```

Each thread does $2 \cdot \text{Width}$ global loads for 2 FLOPs each step — terrible compute-to-memory ratio.

Tiling outline (the 6-step pattern)

1. Identify a tile of global memory accessed by multiple threads.
2. Cooperatively load tile into shared memory.
3. `__syncthreads()` so all loads are visible.
4. All threads consume tile from shared memory.
5. `__syncthreads()` so consumption is done.

6. Move to next tile.

Tiled SGEMM kernel

```
__global__ void TiledMatMul(float *M, float *N, float *P, int Width) {
    __shared__ float Mds[TILE_WIDTH][TILE_WIDTH];
    __shared__ float Nds[TILE_WIDTH][TILE_WIDTH];

    int bx = blockIdx.x, by = blockIdx.y;
    int tx = threadIdx.x, ty = threadIdx.y;
    int Row = by * TILE_WIDTH + ty;
    int Col = bx * TILE_WIDTH + tx;

    float Pvalue = 0;
    for (int p = 0; p < Width/TILE_WIDTH; ++p) {
        // Load one M and one N element per thread
        Mds[ty][tx] = M[Row*Width + p*TILE_WIDTH + tx];
        Nds[ty][tx] = N[(p*TILE_WIDTH + ty)*Width + Col];
        __syncthreads();

        for (int k = 0; k < TILE_WIDTH; ++k)
            Pvalue += Mds[ty][k] * Nds[k][tx];
        __syncthreads();
    }
    P[Row*Width + Col] = Pvalue;
}
```

Tile-size considerations

TILE_WIDTH = 16: block = 256 threads. Each phase: $2 \cdot 256 = 512$ global loads, $256 \cdot 2 \cdot 16 = 8192$ mul/adds $\Rightarrow$ **16 FLOPs / memory load**.

Shared memory per block: $2 \cdot 16 \cdot 16 \cdot 4 =$ **2 KB**. With 16 KB SM: up to 8 blocks.

TILE_WIDTH = 32: block = 1024 threads. Each phase: 2048 loads, 65536 mul/adds $\Rightarrow$ **32 FLOPs / memory load**.

Shared memory per block: $2 \cdot 32 \cdot 32 \cdot 4 =$ **8 KB**. With 16 KB SM: only 2 blocks active.

Arbitrary (non-multiple) matrix sizes — Boundary handling

Tests are different for loading vs computing:

- **Loading M tile:** if (Row < Height && p*TILE_WIDTH + tx < Width) load else write 0
- **Loading N tile:** if (p*TILE_WIDTH + ty < Height && Col < Width) load else write 0
- **Writing P:** if (Row < Height && Col < Width) write Pvalue

Writing 0 makes the multiply-add a no-op. A thread that doesn't compute a valid P element can still participate in loading.

3 Lecture 10 — Memory Coalescing

Core idea

DRAM is slow at random access but fast at burst access. When all threads in a warp request consecutive addresses inside the same **burst section**, one DRAM request serves all of them. Otherwise multiple requests, wasted bandwidth.

DRAM mechanics

- Each DRAM cell = capacitor + transistor. Reading is slow (sense amps, row open).
- **Bursting**: one row read fills an internal buffer; data is streamed out in N transfers at interface speed. DDR2/3: core speed $\frac{1}{4}$ or $\frac{1}{8}$ of interface; buffer width $8\times$ interface.
- **Banks**: multiple banks in parallel reduce dead time on the interface.
- **Modern HBM2/HBM3** stacks DRAM dies vertically with TSVs; way more bandwidth than GDDR (A100 = 1555 GB/s, H100 = 3 TB/s).

Coalescing rule

A warp's load is **coalesced** when the index has the form:

$$A[(\text{expression independent of threadIdx.x}) + \text{threadIdx.x}]$$

i.e., consecutive threads access consecutive addresses.

Worked example: matrix multiply

A 2D C array $M[i][j]$ is stored row-major: $M[i \cdot \text{Width} + j]$.

- **N tile load**: $N[k \cdot \text{Width} + \text{Col}]$ where $\text{Col} = \text{bx} \cdot \text{TILE_WIDTH} + \text{tx} \Rightarrow$ as tx varies, addresses are consecutive $\Rightarrow$ **coalesced**.
- **M tile load**: $M[\text{Row} \cdot \text{Width} + k]$ where $\text{Row} = \text{by} \cdot \text{TILE_WIDTH} + \text{ty} \Rightarrow$ as tx varies, address doesn't change (it changes with ty) — depends on thread mapping; in the basic kernel it's also coalesced when tx is the fast-varying dimension, but column-walking patterns (e.g., $A[\text{tid}] \rightarrow A[\text{tid} \cdot \text{Width}]$) would not be.

Tiling also helps coalescing. Even if the algorithmic access pattern on global memory looks non-coalesced for one operand, you can load a tile coalesced (each thread loads one element at the same relative position) into shared memory and then use it from shared, where coalescing rules don't apply.

4 Lecture 11 — Histogram, Atomics, Privatization

Core idea

Bin each input element by reading it, computing the bin, and incrementing a counter. The naive parallel version has a data race on each counter (two threads read 0, both write 1).

Two partitioning schemes

- **Sectioned**: thread i handles a contiguous chunk of input. Adjacent threads touch distant memory $\Rightarrow$ **not coalesced**.
- **Interleaved**: all threads process one contiguous stride together, then move to the next stride. Adjacent threads touch adjacent memory $\Rightarrow$ **coalesced**. This is the right choice on a GPU.

Data race — concrete example

$$\text{Old} \leftarrow \text{Mem}[\text{x}]; \text{New} \leftarrow \text{Old} + 1; \text{Mem}[\text{x}] \leftarrow \text{New}$$

Run on two threads, $\text{Mem}[\text{x}]$ starts at 0. Possible final values: **1 or 2**, depending on interleaving. Need atomic.

Atomic operations

A hardware-enforced read-modify-write on a single memory location. Other threads attempting RMW on the same address are queued. **Serialized** per location.

```

int atomicAdd(int *address, int val);    // returns the OLD value
// Variants: atomicSub, atomicMin, atomicMax, atomicInc, atomicDec,
//           atomicExch, atomicCAS, atomicAnd, atomicOr, atomicXor
// Scopes:   atomicAdd_system (CPU+all GPUs), atomicAdd (device-wide),
//           atomicAdd_block (within block)

```

Basic histogram kernel (interleaved, no privatization)

```

__global__ void histo_kernel(unsigned char *buf, long size,
                             unsigned int *histo) {
    int i = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x * gridDim.x;
    while (i < size) {
        int pos = buf[i] - 'a';
        if (pos >= 0 && pos < 26)
            atomicAdd(&histo[pos/4], 1);
        i += stride;
    }
}

```

Why atomic performance is bad on global memory

Each RMW takes a read-latency (few hundred cycles) + write-latency. While the op is in flight, no one else touches that location. If many threads contend on the same bin:

$$\text{Throughput} \approx \frac{1}{\text{latency}} \ll \text{peak bandwidth}$$

Latency hierarchy:

Global DRAM atomic: ~1000+ cycles. **L2 atomic:** ~1/10 of DRAM. **Shared memory atomic:** very short, private to block. **Shared atomics** are ~100× faster than DRAM.

Privatization

Instead of all blocks slamming the final histogram, each block builds a **private** histogram in shared memory, then merges into the final histogram once.

```

__global__ void histo_kernel(unsigned char *buf, long size,
                             unsigned int *histo) {
    __shared__ unsigned int histo_private[7];
    if (threadIdx.x < 7) histo_private[threadIdx.x] = 0;
    __syncthreads();

    int i = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x * gridDim.x;
    while (i < size) {
        atomicAdd(&histo_private[(buf[i] - 'a')/4], 1);
        i += stride;
    }
    __syncthreads();

    if (threadIdx.x < 7)
        atomicAdd(&histo[threadIdx.x], histo_private[threadIdx.x]);
}

```

When privatization works

The reduction must be **associative and commutative** (histogram add is). Private copy must **fit in shared memory**. If histogram is too large, partial privatization (privatize hot bins, fall through to global for cold ones).

5 Lecture 12 — Multi-GPU Programming

Why multi-GPU

More compute, more memory than one GPU, better perf/W by amortizing CPU/server cost.

Managing devices from one CPU thread

CUDA calls go to the *current* GPU. `cudaSetDevice(i)` changes it. You can switch while async work is in flight — both GPUs run concurrently:

```
cudaSetDevice(0); kernel<<<...>>>(...);  
cudaSetDevice(1); kernel<<<...>>>(...); // GPU 0 still running
```

Peer-to-peer (P2P) access

```
int can; cudaDeviceCanAccessPeer(&can, dev_X, dev_Y);  
cudaDeviceEnablePeerAccess(peer_dev, 0);  
cudaMemcpyPeerAsync(dst_addr, dst_dev, src_addr, src_dev,  
                    num_bytes, stream);
```

With P2P enabled, transfer goes along the shortest PCIe path — no staging through host. Without P2P, driver routes through host memory.

P2P throughput on PCIe gen2: ~6.6 GB/s simplex, ~12 GB/s duplex (5 GB/s through host). gen3 doubles those.

1D decomposition example — Left-Right exchange

Each subdomain has neighbors on left/right. Do exchanges in two stages:

- **Stage 1:** every GPU sends right, receives from left.
- **Stage 2:** every GPU sends left, receives from right.

Concurrent peer copies in each stage with no PCIe link contention. For 4 GPUs through PCIe switches: ~15 GB/s aggregate.

NUMA effects

On dual-IOH systems, "remote" CPU-to-GPU transfers go through the QPI link, which costs an extra hop:

Local D2H: ~6.3 GB/s. **Remote D2H:** ~4.3 GB/s.

Also: **dual-IOH systems prevent P2P across IOHs** — copies still work but are staged through host. Pin CPU threads to the "near" socket using `numactl`, `GOMP_CPU_AFFINITY`, etc.

6 Lecture 13 — GPU Sharing via MPS (slides 1–25)

Core idea

Multi-Process Service (MPS) lets multiple CUDA processes share *one* GPU context, so kernels and memcopies from different processes can overlap on the GPU instead of being serialized at context-switch boundaries.

Why you'd want it

A typical MPI-style strong-scaling run: as you add MPI ranks, each rank gets less GPU-parallelizable work. Without MPS, each rank has its own context, and the GPU serializes them (context switches). With MPS + Hyper-Q, ranks share one context and can be overlapped.

Requirements

- Linux only.
- Unified Virtual Addressing (UVA).
- Tesla GPU with compute capability ≥ 3.5 , CUDA toolkit 5.5+.
- Exclusive-mode restrictions apply to the MPS *server*, not the clients.

Architecture: Hyper-Q and concurrency

Concurrent kernels (Fermi+, CC 2.0): GPU can run multiple kernels if launched to different streams and there are enough free resources for at least one block of the second kernel on some SM. Fermi: up to 16 concurrent; Kepler: up to 32.

Kepler Hyper-Q: 32 hardware work queues, one per stream — no inter-stream false dependencies.

MPS layers on top: one MPS client gets 2 work queues per channel; total still 32. Case 1: streams per client $\leq$ channels (no serialization). Case 2: streams per client $>$ channels (**false dependency / serialization**).

How MPS works

A `nvidia-cuda-mps-control -d` daemon spawns an `nvidia-cuda-mps-server`. All processes started after the server route their CUDA work through the server, which holds the single shared context. Without MPS, two processes on one GPU each get their own context — kernels never overlap, context switches add latency. With MPS, kernels from different processes can interleave.

Setup (single GPU):

```
export CUDA_VISIBLE_DEVICES=0
nvidia-smi -i 0 -c EXCLUSIVE_PROCESS
nvidia-cuda-mps-control -d
```

No application code changes needed.

MPS notes

- MPS does **not** automatically distribute work across multiple GPUs — the app must still handle affinity.
- Profile with `nvprof --profile-all-processes -o file.%p`.

7 Lecture 14 — Dynamic Parallelism (DP)

Core idea

Before Kepler, only the CPU could launch GPU kernels. With Dynamic Parallelism (Kepler+, CC 3.5+), **a GPU kernel can launch its own child kernels** — dynamically, simultaneously, and independently.

What it unlocks

- **Library calls from kernels** (cuBLAS, etc.) inside a kernel.
- **Data-dependent parallelism** — irregular loop bounds like `for j = 1 to x[i]`, where worst-case static grid is wasteful.
- **Recursive parallel algorithms**.
- **Adaptive mesh refinement** — spawn finer grids only where needed (e.g., Mandelbrot regions, fluid simulation hotspots).

- **Batched nested parallelism** — move the outer batch loop to GPU; CPU is freed.

Syntax

Familiar: child launches use the same `kernel<<<grid, block>>>(args)` syntax inside a `__global__` function. Synchronize children with `cudaDeviceSynchronize()` on the device side.

```
__device__ float buf[1024];
__global__ void dynamic(float *data) {
    int tid = threadIdx.x;
    if (tid % 2) buf[tid/2] = data[tid] + data[tid+1];
    __syncthreads();
    if (tid == 0) {
        launch<<<128, 256>>>(buf);
        cudaDeviceSynchronize();    // waits for the child grid
    }
    __syncthreads();                // <-- needed: cudaDeviceSync
                                    //      does NOT imply syncthreads
    cudaMemcpyAsync(data, buf, 1024);
    cudaDeviceSynchronize();
}
```

Basic rules — memorize

- **Launch is per-thread.** If two threads hit the launch line, two child grids spawn.
- **Sync is per-block.** `cudaDeviceSynchronize()` waits for *all* launches by *any* thread in the block — not just the calling thread.
- `cudaDeviceSynchronize()` does **not** imply `__syncthreads()`. If you need warp/block barrier behavior afterward, add `__syncthreads()`.
- **All launches & copies are async.** No synchronous device-side launch.
- CUDA primitives are per-block — cannot pass streams/events to children.
- Constants set from host; textures/surfaces bound only from host; ECC errors reported at host.

Library example pattern

```
__global__ void libraryCall(float *a, float *b, float *c) {
    createData(a, b);
    __syncthreads();
    if (threadIdx.x == 0) {
        cublasDgemm(a, b, c);    // library does its own launch
        cudaDeviceSynchronize(); // wait for that launch
    }
    __syncthreads();            // make sure others see result
    consumeData(c);
}
```

Cross-cutting checklist for the exam

From Midterm 1 patterns that will reappear:

- Compute warps/blocks/threads given launch config. (Reminder: warp = 32 threads.)
- Compute grid dimensions for a 2D image given block size — ceiling division.
- Count divergent warps based on image-boundary blocks.
- Identify which memory accesses are coalesced.

New for Exam 2:

- Reason about UM behavior: pre-Pascal vs Pascal+. What happens on concurrent CPU/GPU access? On a kernel page fault?
- Tiled SGEMM: ratio of FLOPs to global loads as a function of TILE_WIDTH. Shared memory footprint per block.
- Boundary handling for arbitrary matrix sizes: three distinct tests (load M, load N, write P).
- Coalescing on tiled algorithms: which access pattern is coalesced in the tile load?
- Atomics: latency tiers (DRAM $\gg$ L2 $\gg$ shared), why privatization helps, when it does not apply.
- Multi-GPU: P2P throughput numbers, what dual-IOH means for P2P, NUMA local vs remote bandwidth.
- MPS: what problem it solves, the difference vs plain CUDA streams, false-dependency case.
- Dynamic Parallelism: launch per-thread, sync per-block, `cudaDeviceSynchronize()` $\neq$ `__syncthreads()`.

Trivia worth memorizing: Coalescing rule `A[indep + threadIdx.x]`. Atomic returns OLD value. `cudaMallocManaged` replaces both `malloc` and `cudaMalloc`. `cudaMemAdvise` hints don't move data; `cudaMemPrefetchAsync` does.